A PROJECT WRITE-UP · NOVEMBER 2026

Can a company's own filing tell you what its stock will do next?

What I built, what I found, and what went wrong on the way. A study of 150 S&P 500 stocks, 2000 to 2026, using 64,213 SEC filings.

Krishna Harsha Tangirala

Code and data	https://github.com/harshatangirala/sec-filings-price-impact
Interactive dashboard	https://sec-filings-study.streamlit.app/ (free apps go to sleep; if you see a "wake this app up" button, press it and wait a minute)
Models used	ChronoBERT, BGE-base, BGE reranker, Qwen2.5-7B (all on Hugging Face)

The short version

Public companies in the US have to file reports with the SEC. There are yearly ones (10-K), quarterly ones (10-Q), and short "something just happened" ones (8-K) that cover things like earnings, a merger or a CEO leaving. I wanted to know whether you could read one of these the moment it appears and say anything useful about how the stock will react. And a second question sat underneath that one: do the *words* in a filing add anything, once you already know the numbers?

The short answers are these. The reaction is partly predictable, but only the *size* of it. On the final test period the best model scored a macro-F1 of 0.384, against about 0.33 for pure luck. It can tell which filings will move a stock (AUC 0.63) but it is a coin flip on *which way* (AUC 0.49). The text did not help at all: adding language models changed the score by -0.001, with an interval that sits comfortably around zero. And one result surprised me. After very good earnings, stocks did slightly *worse* over the following months, not better. I spent a long time checking whether that was a mistake, and I explain below why I think it is mostly real in this sample, and why I still wouldn't trust it too much.

Most of what follows is about how I got there, because that is where the actual work was. I found several bugs that would have made the results look better than they were, and I'd rather show those than hide them.

1. The idea, and the one rule I kept to

There is an obvious way to do this badly. You take every filing, look up what the stock did around it, fit a model, and get a lovely score. The problem is that you have quietly used information that nobody had at the time. If a filing was published at 4:30 in the afternoon, you can't use that day's

closing price as an input, and you can't trade at that close either. So the one rule I kept to is this: **act like a trader sitting at the screen the moment the filing becomes public**. Every input has to be something that trader could have known, and the thing I predict has to be something they could have actually captured.

I split the work into three tracks and tried hard not to mix them. Track A is an event study: how do prices move around filings, nothing more. That is description, not prediction. Track B is the prediction problem. Track C asks whether the facts a language model pulls out of a filing match what a human would pull out, and it is judged on the text, never on prices. Track C turned out to need hand-labelled examples that I don't have yet, so it only half exists. I'll be clear about that when we get there.

The questions, in the order I wanted to answer them:

1. Can we predict the next reaction at all?
2. Does the text add anything beyond numbers that are already public (price history, company size, accounting)?
3. Does the well-known post-earnings drift show up in this data?
4. Could any of it make money?

I also wrote these down, along with the metric and the pass or fail rules, before looking at the final test. That is called pre-registering, and it is boring, and it is the thing that stops you from changing the question after you've seen the answer.

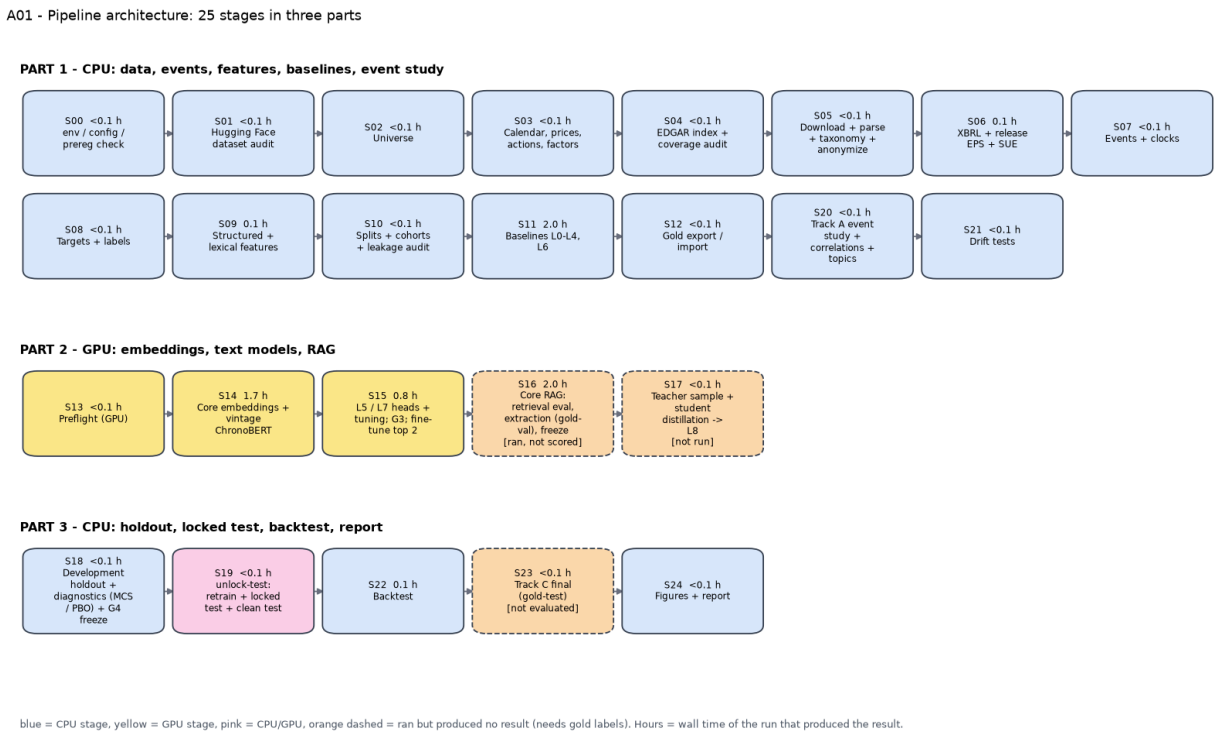


Figure 1. The whole pipeline: 25 stages in three parts. Part 1 ran on my laptop, Part 2 on a Colab GPU, and Part 3 back on CPU. The two orange boxes ran but couldn't produce a result because they need human-labelled data I don't have.

2. The data

Everything comes from free, public sources, and I pinned the version of each so someone else can rerun it. The table lists what I used.

source	what I used it for	how much	access
SEC EDGAR	all 10-K, 10-Q and 8-K filings for the 150 firms, plus exhibits	64,213 filings	public, free
EDGAR XBRL	balance-sheet and income numbers, as first reported	304,348 facts	public, free
Earnings press releases	the EPS figure in each release, to build an earnings surprise	14,981 results releases	parsed from EDGAR exhibits
Daily prices	returns, volatility, the benchmark, the backtest	1,024,405 daily bars	my own price files, filled in with Yahoo Finance
Fama-French factors	to measure what is 'abnormal', and for the alpha test	daily factors	Kenneth French's data library
S&P 500 membership list	to choose the firms	history since 1996	a public GitHub list, pinned to one commit
Loughran-McDonald dictionary	counts of positive and negative finance words	1993–2025 edition	free for academic use
Hugging Face SEC corpus	only to pick firms and to check my parser (never as study text)	10-Ks up to 2020	JanosAudran/financial-reports-sec

I picked 150 companies that were members of the S&P 500 at some point since 2000, using a fixed random seed and spreading them across 12 industries. They came from 743 firms that had prices in my files and at least one 10-K in the Hugging Face corpus. That gave 60,261 **events**: 45,193 8-Ks, 11,271 10-Qs and 3,797 10-Ks. If a firm filed several 8-Ks on the same day I merged them into one event. Events run from January 2000 to July 2026.

I started out planning 500 stocks, 353 from the S&P 500 and 147 that were not. The first half of that run finished. Then, while checking it, I found a bug in how I looked up accounting numbers (more on that later), and rather than re-run everything at the larger size I dropped to the 150-stock version, which I could finish properly. That is why the final study is 150 S&P stocks only, and why I don't compare S&P and non-S&P firms in the end.

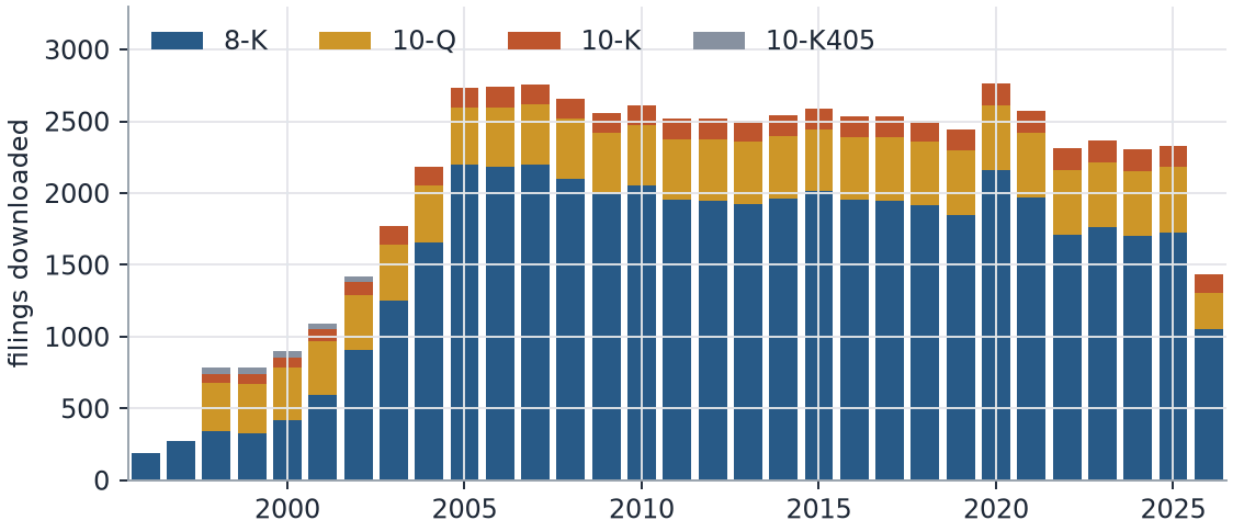


Figure 2. Filings downloaded per year. Quarterly and yearly reports take off in the early 2000s, and most of the volume is 8-Ks.

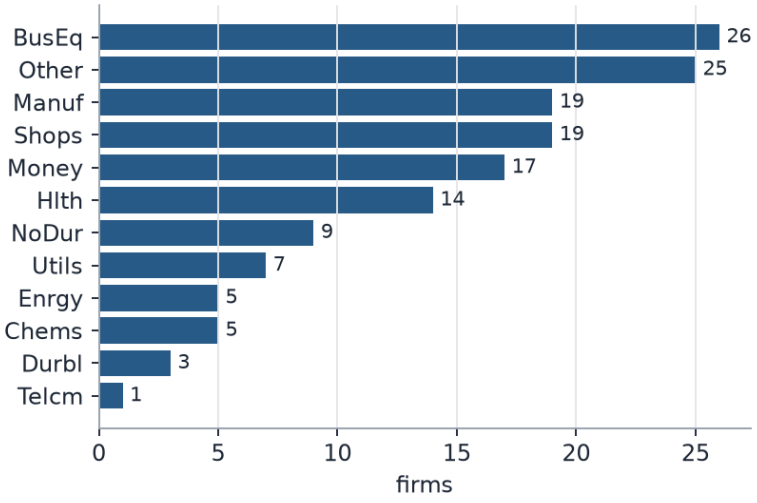


Figure 3. The 150 firms by Fama-French industry. "BusEq" is business equipment (software, chips, hardware) and "Other" is a grab bag.

The weakness I have to flag right here. All 150 firms still exist today. Companies that were delisted, bought, or went bust are not in the sample. That is survivorship bias, and it matters most for the earnings-drift result later. My data covers about a third of the 1,094 tickers that were ever in the index during these years, and I couldn't fix that with free price data.

3. Turning filings into text a model can read

The raw filings are messy HTML, sometimes ancient SGML, full of tables and page numbers. Getting from there to clean text was a large part of the first week. I'll go through what happens, in order.

Download and clean. I downloaded 64,213 filings from EDGAR politely (a few requests a second, with my contact details in the header, as the SEC asks). 63,933 parsed cleanly, and 280 had none of the sections I wanted. I turned HTML into plain text and flattened tables into rows, since the earnings numbers I wanted are in tables. That meant flattening 2,550,579 of them.

Keep only the parts that carry news. A 10-K is mostly boilerplate. I kept the business description, risk factors, MD&A and market-risk sections from 10-Ks; MD&A, market-risk and risk-factor sections from 10-Qs; and the item text plus the press-release exhibit from 8-Ks. Finding these sections is harder than it sounds, because the table of contents repeats every heading. My rule was to take the candidate start that has the longest span before the next heading. That left 121,398 sections.

Hide the company's name. I replaced the firm's own names, former names and ticker with a placeholder, 881,908 name mentions and 249,224 ticker mentions in all. Without this, a model can quietly learn "this is Apple" instead of "this text sounds bad", and that is a form of leakage.

Drop number-heavy lines, cap the length, and cut into chunks. Each section was chopped into 350-word windows that overlap by 50 words, giving about 900,694 chunks, roughly 474 million tokens. The same chunks went to every model.

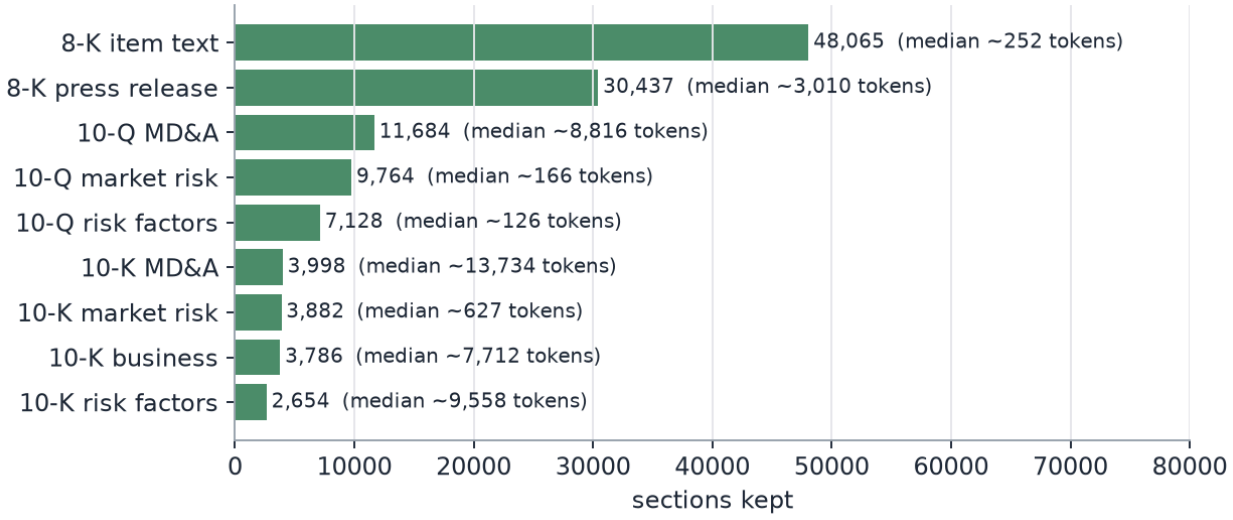


Figure 4. How many sections of each type I kept, and how long a typical one is. 10-K sections are long (thousands of tokens), 8-K item text is short.

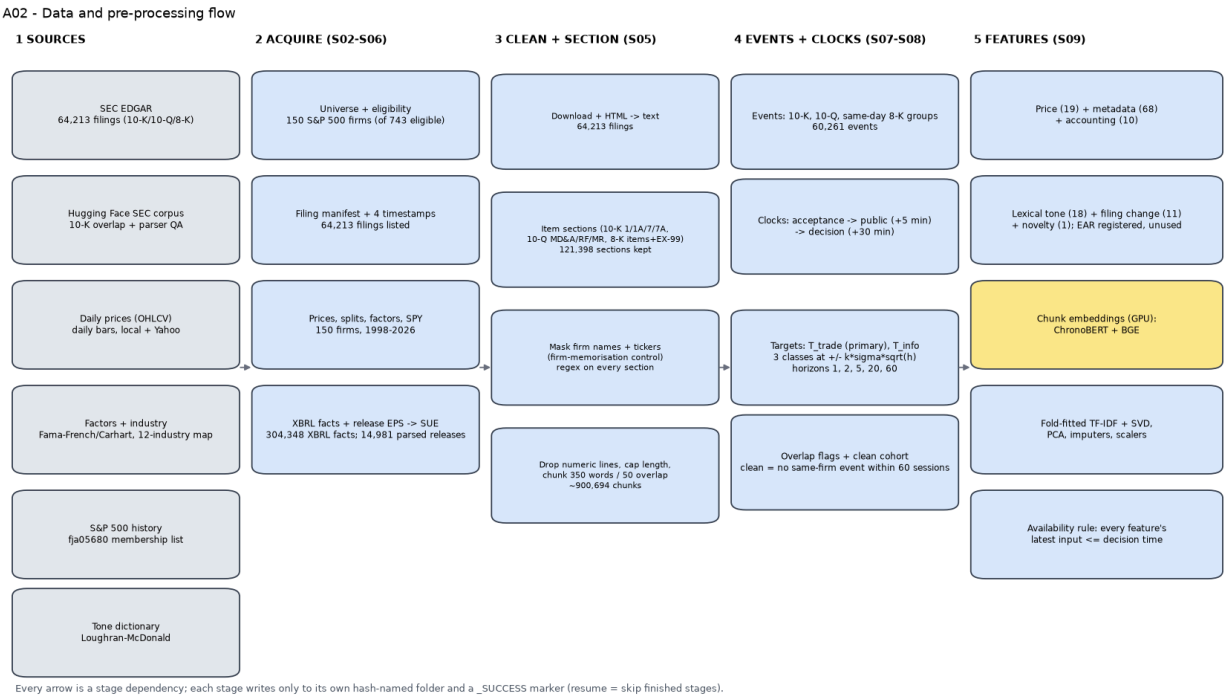


Figure 5. From sources to features. The blue boxes ran on my laptop, the yellow one needed the GPU.

I also built some old-fashioned word features, because I wanted a fair baseline for the fancy models to beat. These are counts of positive and negative finance words (Loughran-McDonald), how much a filing's language changed since the same firm's previous one, and how much of it was new. None of these learn a vocabulary from the future. The change features use a fixed hashing trick, and where I did fit something like TF-IDF, I refitted it inside each training fold only.

4. What counts as "the stock reacted"

Every event carries four timestamps: when the SEC accepted the filing, when I assume it was public (five minutes later), when a trader could decide (thirty minutes after that), and the next time the market opens. Exact acceptance times come from the filing header for 96% of events. The SEC's own JSON feed turned out to follow two different time conventions mixed together, which I only noticed because I checked it against the headers on a sample. The five and thirty minute gaps are assumptions, and I say so wherever the results appear.

The thing I predict is the stock's **abnormal return** in the first tradable session: the stock's return from the open to the close, minus what its market sensitivity says it should have done given what the S&P 500 did over exactly the same hours. I turn that into three classes. A move bigger than half the stock's own typical daily swing is Up or Down, and anything smaller is Sideways. Using each stock's own volatility keeps quiet utilities and wild tech names comparable. The result is roughly 29% Down, 41% Sideways and 30% Up.

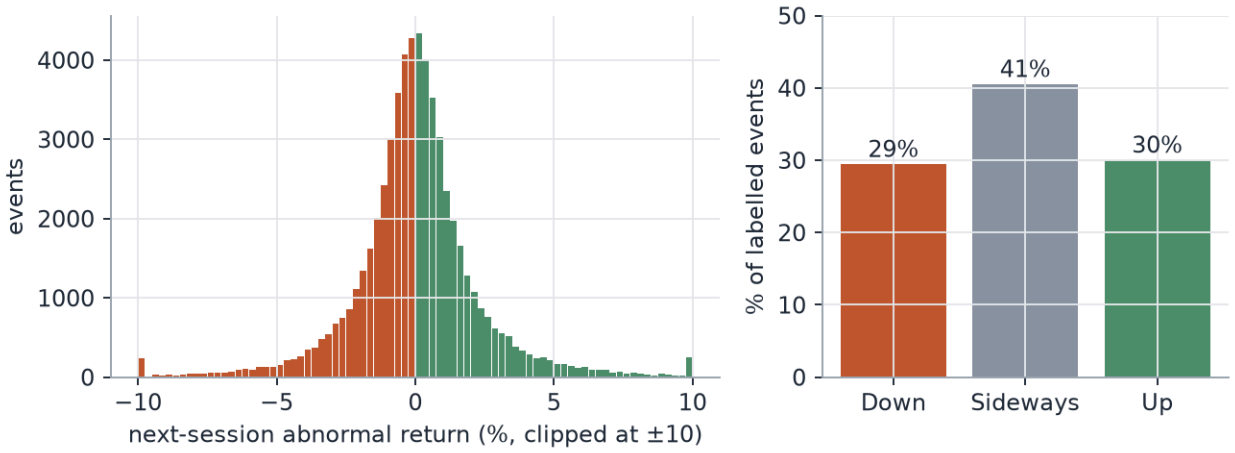


Figure 6. Left: abnormal returns on the next session cluster tightly around zero, with fat tails. Right: how the three classes come out.

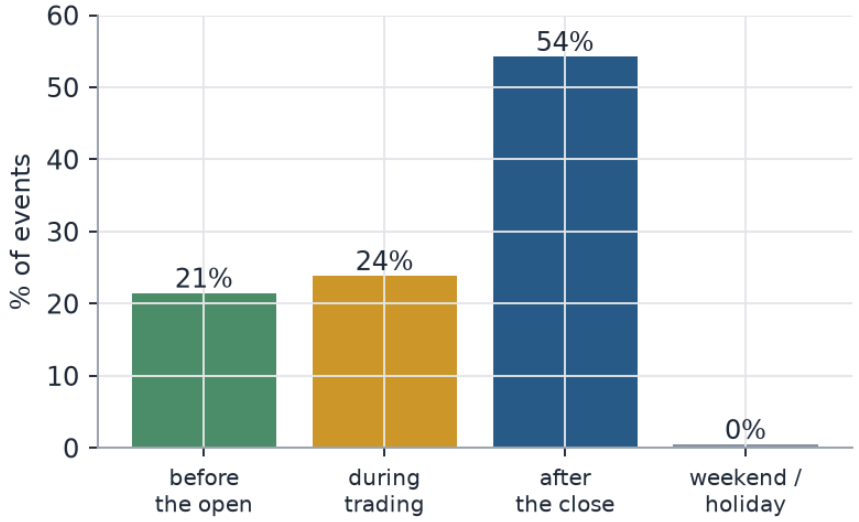


Figure 7. More than half of filings land after the market closes. That is why the trade happens at the next open rather than at the filing time.

5. The features, and the leaks I went looking for

Each event gets 128 features in seven groups: 19 about price and market state, 68 metadata (what type of event, which 8-K categories, which sections exist, the industry, the calendar, the time of day), 10 accounting numbers including an earnings-surprise score, 18 word-tone counts, 11 change-since-last-filing measures, and one novelty score. Each feature carries the latest time of any information it used, and a check runs on every single row that this time is no later than the decision time. The count of violations was 0.

The earnings surprise (SUE) deserves a sentence. I don't have analyst forecasts, so I defined "surprise" as this quarter's earnings per share minus the same quarter last year, divided by how variable that gap has been in the past. It is a crude version of the real thing, and I'll come back to why that matters.

Then I tried to catch myself cheating. I wrote a list of 24 leakage checks, each with an assertion in the code, and planted four deliberate leaks (a feature built from future returns, an earnings reaction used for its own event, a surprise taken from a later filing, and a future document slipped into retrieval) to see whether the audit would notice. All four were caught. More usefully, the checks caught three real problems of my own while I was building:

- **Same-day 8-Ks.** When several 8-Ks arrived on one day, I had clocked the merged event at the first filing, so text and numbers from later filings leaked in. An assertion failed, and I moved the clock to the last acceptance.
- **A 10-Q hiding in an earnings surprise.** A quarterly report filed minutes after the earnings release was feeding that release's own surprise history. Another assertion caught it.
- **The stale balance sheet.** The worst one, and it deserves its own paragraph below.

A filing reports this year's balance sheet and last year's at the same moment. My lookup for "the latest total assets known at decision time" sorted by publication time only, so it picked between the two more or less by accident. It didn't look into the future, so no alarm went off. But I found it by comparing the 150-stock run with the earlier 500-stock run on the same firms: events and returns matched exactly while six accounting features did not, in a third to a half of rows. Checked against the raw data, the old code returned last year's figure in 52% of a random sample of events. After the fix it was wrong in 0 of 1,500. This is what made me discard the earlier 500-stock results.

6. How I tested it

A random split would be wrong here, because market conditions in 2015 tell you about 2014 and 2016. So I only ever train on the past. The pipeline trains on everything up to 2008 and tests on 2008-2009, then trains up to 2010 and tests on 2010-2011, and so on for six two-year folds ending in 2019. The 2020 to 2022 period is a separate development set that I used to choose the final model. The 2023 to July 2026 period is the locked test. I scored it once, after the final model was frozen and its settings were hashed. Events whose outcome window crossed a boundary are dropped, and there is a 90-day gap between blocks.

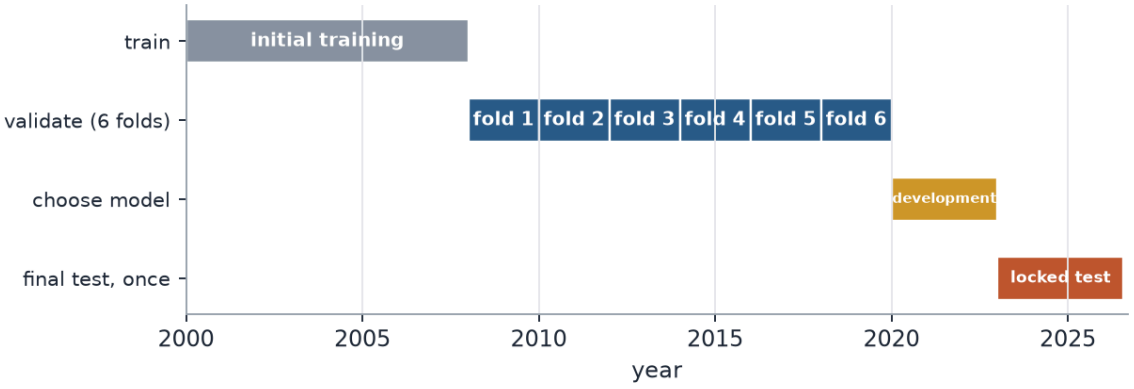


Figure 8. How the 26 years were used. The first folds start in 2008 because the model needs at least eight years of history to learn from.

The metric. My main number is macro-F1: the average of how well the model does on Down, Sideways and Up, weighting the three equally. It stops a model scoring well just by predicting the biggest class. I also report accuracy, AUC (how well the probabilities rank events), log-loss and calibration error (do the stated probabilities match reality), and every interval comes from resampling whole months, since events in the same month share market shocks.

What a score of 0.38 means. This tripped me up at first. The report originally compared scores to a "majority class" baseline of 0.19, which makes everything look great. But a model with balanced predictions gets about 0.33 by luck on three roughly equal classes. I measured that directly by shuffling a model's own predictions against the true labels (0.33), so that is the line to read scores against.

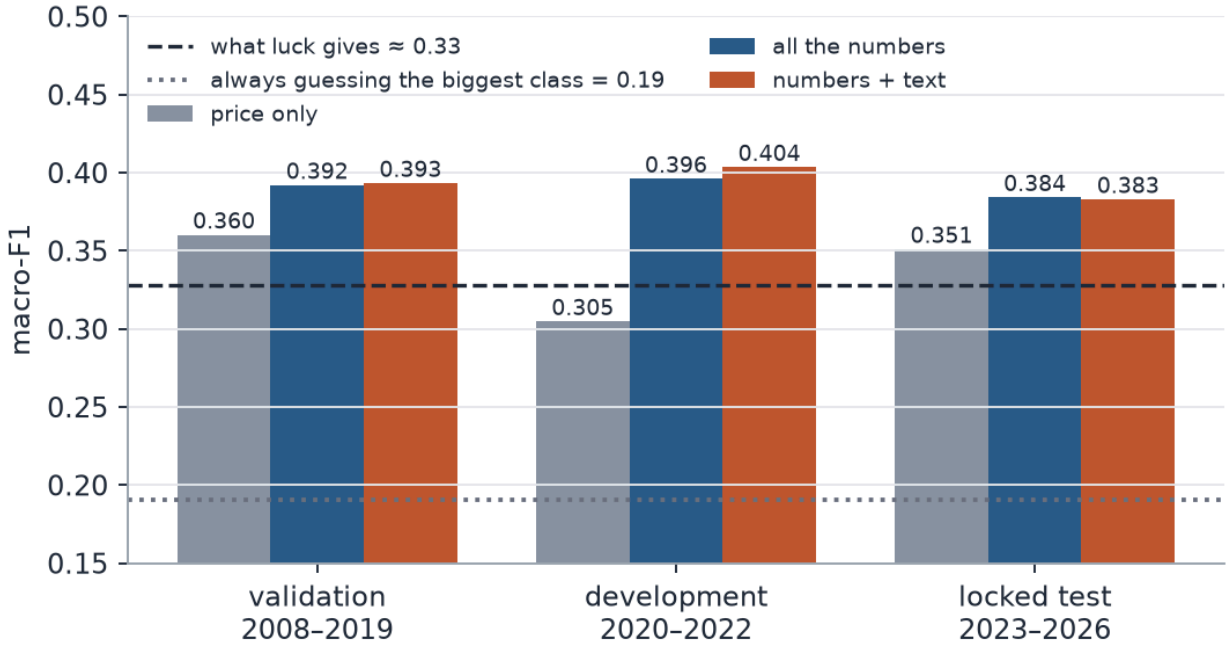


Figure 9. Macro-F1 for three models in three periods. The dashed line is what luck gives you.

7. The models: what I trained, what I borrowed

There are two quite different kinds of models in this project, and I don't want to blur them. The ones that make the predictions are small, classical models that I trained on my own data. The big neural models are borrowed and frozen: I use them only to turn text into numbers or to answer questions, and I never changed their weights.

model	status	size (approximate)	why I picked it	where to find it
LightGBM	trained here	tree ensemble; the all-numbers model used about 25 leaves per tree	fast on tables with lots of mixed features	—
CatBoost	trained here	tree ensemble	a second, different tree method as a cross-check	—
Elastic net	trained here	about 400 weights	simple and convex, so it can't hide overfitting	—
ChronoBERT (26 yearly versions)	borrowed, frozen	about 150 million each	each version only saw text up to its own year	Hugging Face
BGE-base-en-v1.5	borrowed, frozen	about 110 million	strong general text encoder; also does the passage search	Hugging Face
BGE reranker-base	borrowed, frozen	about 280 million	puts the best retrieved passages first	Hugging Face
Qwen2.5-7B-Instruct	borrowed, frozen	about 7.6 billion	open model I can run myself, no paid API	Hugging Face
SEC-BERT	planned, dropped	about 110 million	trained on 10-Ks; left out to save GPU time	Hugging Face
FinBERT-tone	planned, dropped	about 110 million	finance tone classifier; left out to save GPU time	Hugging Face

For the trained models I used LightGBM, CatBoost and an elastic-net logistic regression, because they are the standard tools for tables of mixed features and they are quick to tune. Using three different ones was a cross-check as much as anything else: if all three agree, the result is not an accident of one library. The elastic net was a good conservative baseline. Its optimisation problem is convex, so I could even verify it found the global optimum, which you can't say for trees. It ended up being the best model on the all-numbers rung.

I tuned settings with Optuna on the three most recent validation folds, then refit on every fold. The class weights balance the three classes and also down-weight events that overlap in time. Predicted probabilities are corrected back to the real class frequencies, and the final class is whichever has the highest probability relative to its frequency.

One honest observation about tuning. I expected it to matter more than it did. Across 32 combinations of rung and model, the median change in the validation score from tuning was +0.000, and only 44% improved at all. The few big improvements were the elastic net, whose default settings were bad. The search budgets were also tiny (6 to 15 trials), smaller than the ten random starts Optuna uses by default, so several rungs ended up with identical settings. I would not call any of these models "fully tuned".

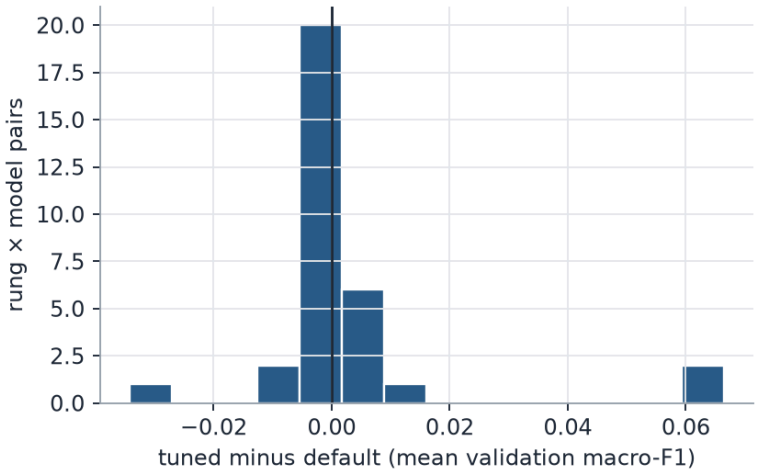


Figure 10. Almost nothing changed when I tuned. A few elastic-net models improved a lot, which is the right-hand bump.

The experiment is built as a ladder, so I can see exactly what each kind of information adds. Each rung gets the one before plus something new, always on identical events.

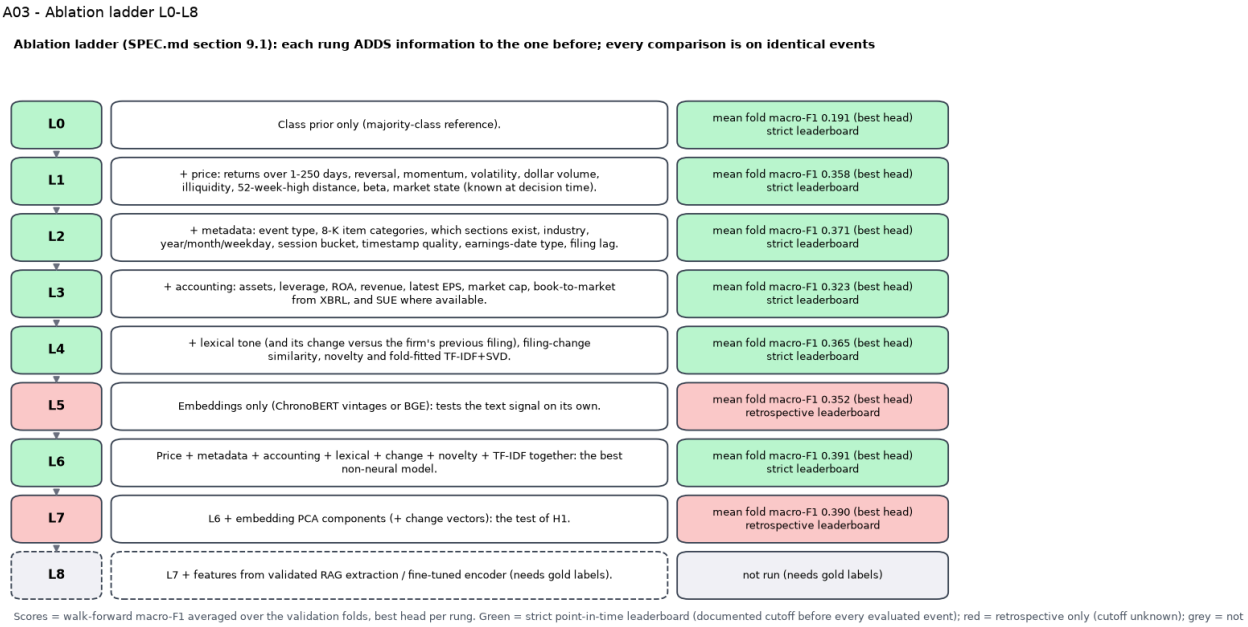


Figure 11. The ablation ladder. Green rungs only use information that was clearly available at the time, red ones use a model whose training cutoff is unknown.

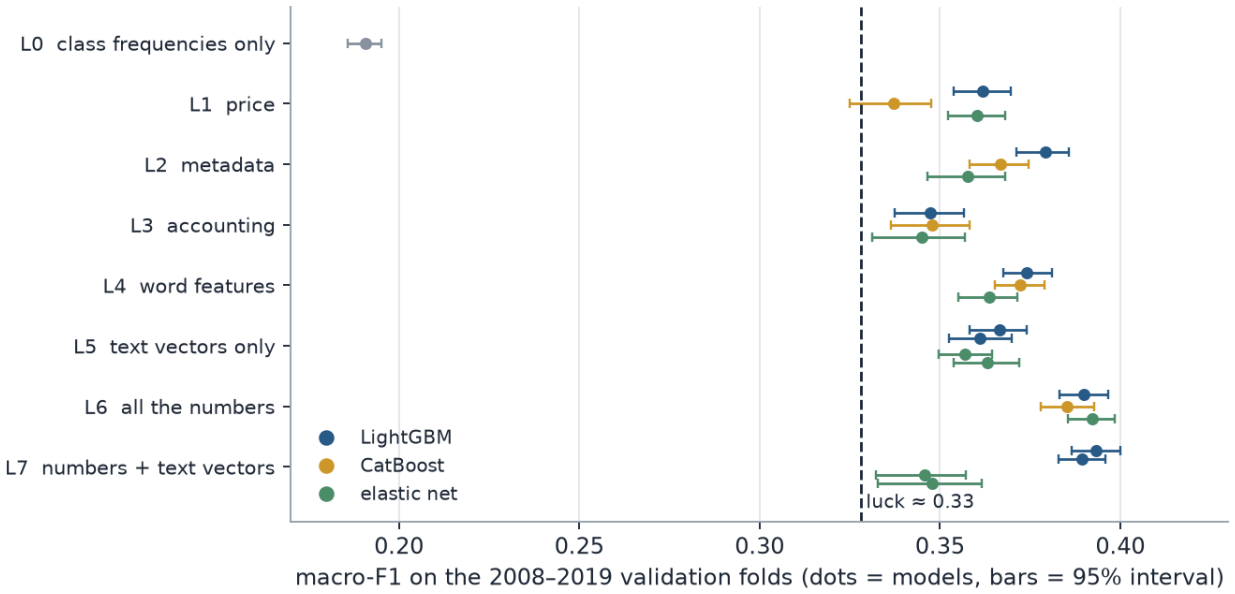


Figure 12. Validation results for every rung and model. Metadata (L2) is the strongest single group. Accounting alone (L3) is the weakest, and putting everything together (L6) is best.

Two rungs surprised me. Text vectors on their own (L5) come close to the word features, a little below them, so the language models do contain some real signal. And the elastic net got much *worse* when I added the text vectors to everything else (0.39 down to 0.35). A regularised linear model drowning in hundreds of correlated extra columns is not a shock in hindsight, but it was a good reminder that more features is not more information.

8. The pre-trained models, and where the embeddings come from

People sometimes ask where the embeddings were downloaded from. They weren't. I downloaded the *models* from Hugging Face, at fixed versions, and ran my own filing text through them on a Colab GPU. The embeddings exist only for my 60,030 filing events and never saw a price.

ChronoBERT is the one I'm happiest with. It comes as 26 yearly versions, and each was trained only on text written up to its date. For an event in year Y I use the version trained to the end of year Y-1, so the encoder cannot know anything from after the filing. If a model has read the future, it can look clever for the wrong reason, and this removes that worry. **BGE-base** is a strong general-purpose encoder with no stated cutoff, so it only counts on the less strict leaderboard. It also drives the passage search I describe later.

Each of the 900,000 chunks becomes 768 numbers. I averaged and took the maximum over a section's chunks, then combined sections into one vector of 1,536 numbers per event, and also stored how far the vector moved compared with the same firm's previous filing of the same type. Before using them I shrank the vectors with PCA, fitted inside each training fold. Embedding took 56 minutes with ChronoBERT and 39 minutes with BGE on the Colab GPU.

Two other encoders, SEC-BERT and FinBERT, were in my original plan. I dropped them to fit the GPU budget. That was a time decision, not a result: I have no evidence they would have done worse, and a domain model like SEC-BERT might have done better.

9. Fine-tuning: what I planned and why I didn't do it

I had a fine-tuning stage in the plan. The idea was to take the two best encoders and train them end to end on my labels, with a grid over learning rate, batch size and weight decay. But I made it conditional. Fine-tuning is expensive, and it multiplies the number of configurations you try, which is how people fool themselves. So the rule was: fine-tune only if adding text to the numbers improves the score by at least 0.01, with an interval that excludes zero.

It didn't come close. Across the validation folds the change was −0.001 with an interval from −0.007 to +0.005, and on the development and locked periods the estimates were +0.007 and −0.001, neither with an interval that excluded zero. So the gate stayed closed and **no encoder was fine-tuned**. The honest label for the language side of this project is "frozen embeddings, plus a retrieval and extraction pipeline", not "fine-tuned models". If you do want to see what fine-tuning would have looked like, the code is in the repository and runs on a smoke test, but it has never run on the real data.

10. The RAG experiment, and why its results are not in the results

The other use of a language model was to pull specific events out of a filing: did the company give guidance, record an impairment, announce a restructuring, a buyback, a dividend, a merger, a lawsuit, a change of management. Embeddings blur all of that into one vector, and I wanted facts I could check.

The pipeline works like this. A filing's text is cut into chunks, and I search them two ways at once, by keywords and by embedding similarity, then merge the two rankings and let a reranking model sort the top thirty. The best eight passages go to Qwen2.5-7B, which answers twelve questions in a fixed JSON format with a quote and a chunk id for every answer. Then a set of guards checks the answer. Every quote has to appear word-for-word in the chunk it cites, any number has to appear in its evidence, and anything that fails is blanked out.

It ran on 149 filings from the development period, at about 47 seconds each, two hours in total. And that is where the story stops. To say whether it is any good I need human-labelled examples to compare against, and I haven't made them. The templates are pre-filled and ready, but I haven't annotated them. So I can't report accuracy for the extraction, I couldn't test whether the extracted events improve the predictions, and I could not train the smaller student model I'd planned. The chart below only shows how often the model said each event was present. Treat it as a description of what the system produces, not as evidence that it works.

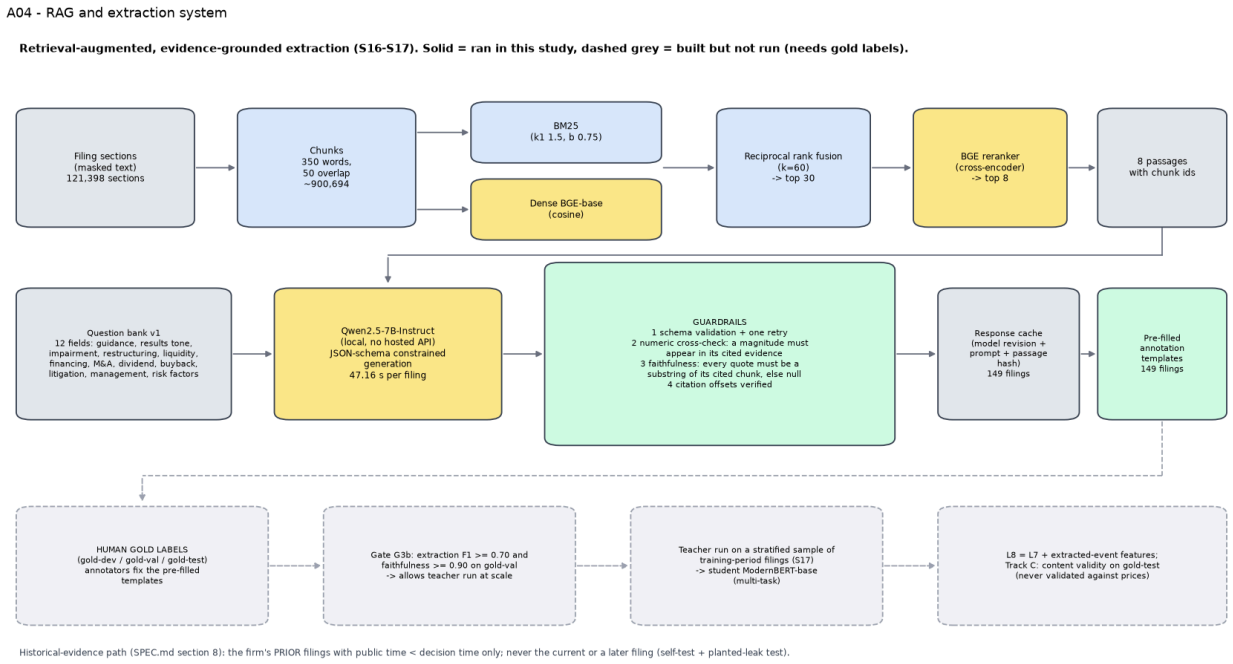


Figure 13. The retrieval and extraction system. Solid boxes ran. Dashed grey boxes are built but waiting on human labels.

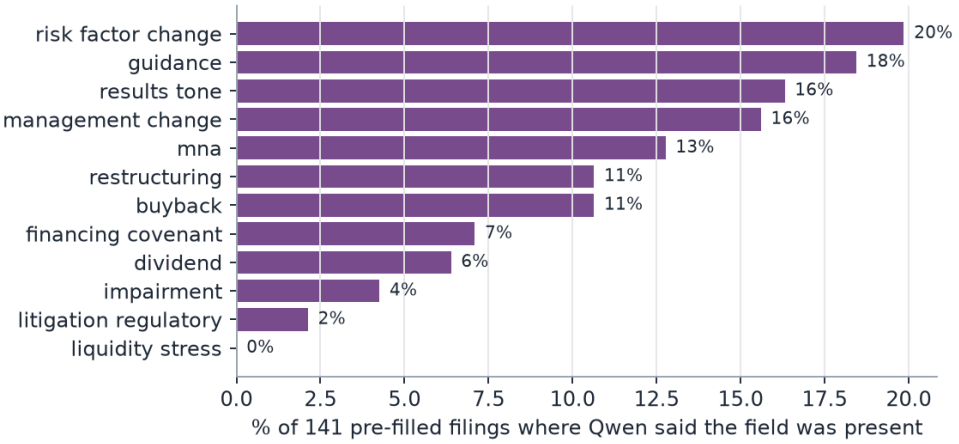


Figure 14. How often Qwen said each field was present in 141 filings that returned a valid answer (8 returned nothing usable). Unchecked output.

11. Results

This is the main table. "All the numbers" is the best model with no text, and "numbers + text" adds the embeddings. The development and locked periods are the ones where the model never saw the data it was scored on.

model	period	events	accuracy	macro-F1 (95% interval)	AUC	log-loss	calibration error
price only	validation 2008–19	28,533	0.366	0.360 [0.352, 0.368]	0.549	1.083	0.018
all the numbers	validation 2008–19	28,533	0.416	0.392 [0.385, 0.398]	0.591	1.069	0.005
numbers + text	validation 2008–19	28,533	0.413	0.393 [0.387, 0.400]	0.587	1.067	0.008
price only	development 2020–22	7,453	0.359	0.305 [0.286, 0.322]	0.572	1.078	0.024
all the numbers	development 2020–22	7,453	0.428	0.396 [0.383, 0.408]	0.605	1.060	0.011
numbers + text	development 2020–22	7,453	0.428	0.404 [0.391, 0.415]	0.602	1.060	0.011
price only	locked test 2023–26	8,199	0.361	0.351 [0.339, 0.363]	0.526	1.097	0.040
all the numbers	locked test 2023–26	8,199	0.401	0.384 [0.375, 0.394]	0.590	1.071	0.009
numbers + text	locked test 2023–26	8,199	0.403	0.383 [0.371, 0.395]	0.590	1.071	0.028

A few things to take from it. The scores are modest but real, and they hold up: validation, development and the final test all land around 0.38 to 0.40. The calibration error is small for the all-numbers model (0.009 on the locked test), so when it says 40%, something like 40% happens. And the log-loss is only slightly better than just using the class frequencies (about 1.096), which is a fair summary of how much there is to find here.

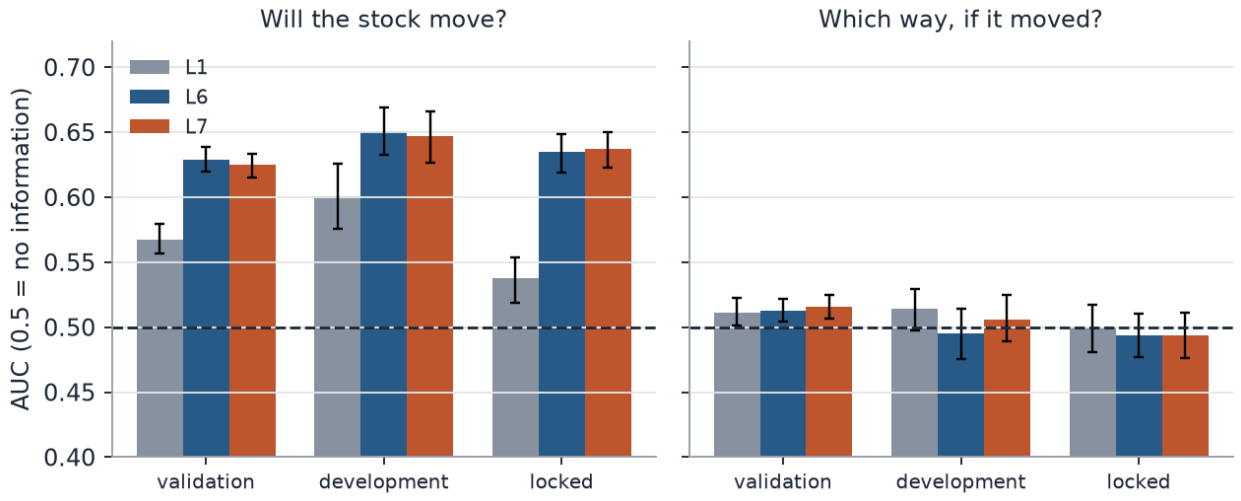


Figure 15. The most important chart in the project. The model is good at telling which events will produce a big move, and no better than a coin at telling which direction. An AUC of 0.5 means no information.

That split explains what the model is actually doing. Metadata like event type, time of day, and the stock's recent volatility predict how much the stock will react. An earnings release after the close for a volatile stock is going to move, whatever the content. Whether the move is up or down depends on whether the news is better or worse than expected, and my numbers contain almost nothing about that. The accounting-based surprise is the nearest thing, and as the earnings section shows, it behaves strangely.

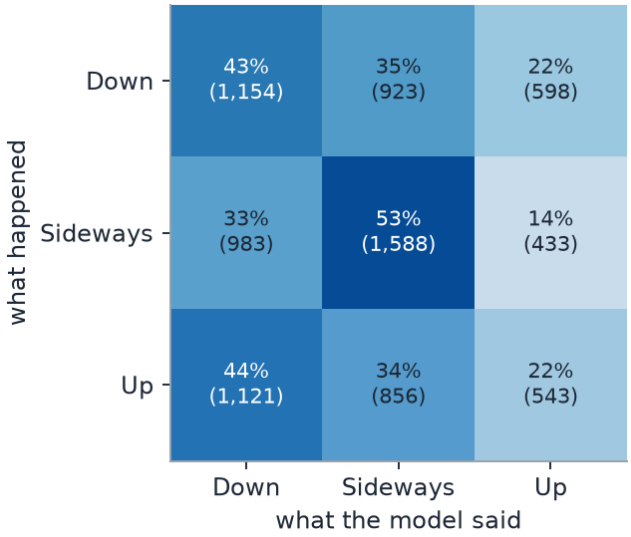


Figure 16. The all-numbers model on the locked test. It finds the Sideways events best (53%) and does little better than chance on telling Down from Up.

Did the text help? No. The comparison I care about is the same model with and without the language-model embeddings, scored on identical events. On the locked test, adding text changed macro-F1 by -0.001 (95% interval -0.012 to +0.010). On the earlier folds and the development period it was equally flat. The yearly ChronoBERT versions did not behave differently from the single BGE model either, and a stacked ensemble did not beat the best single model. The numbers beat price alone by about 0.03 in the validation and locked periods, so the structured features are doing real work. The text is not.

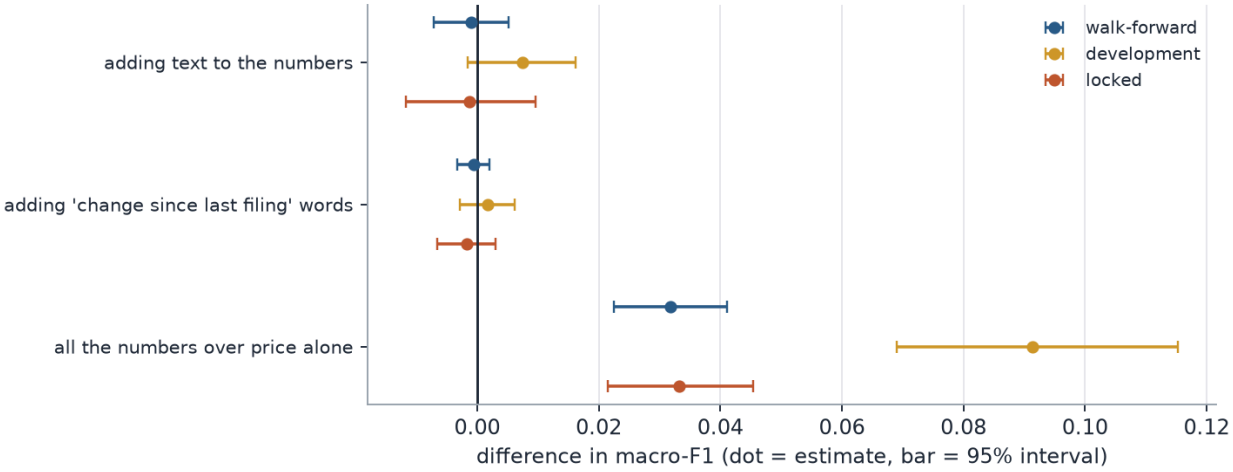


Figure 17. Every comparison I pre-registered, with its interval. Where the bar crosses the vertical line, I can't tell the difference from zero.

12. The earnings drift puzzle

The classic finding in this area is that stocks keep drifting in the direction of an earnings surprise for weeks afterwards. Good news, then more gains. So I sorted earnings reports into five groups by the size of the surprise and tracked the average abnormal return for the next 2 to 60 trading days.

It went the other way. On announcement day it behaves the way you would hope: the best-news group rose 0.81% and the worst-news group fell 0.54%. Then, over the following months, the best-news group *lost* 1.85% and the worst-news group *gained* 1.40%.

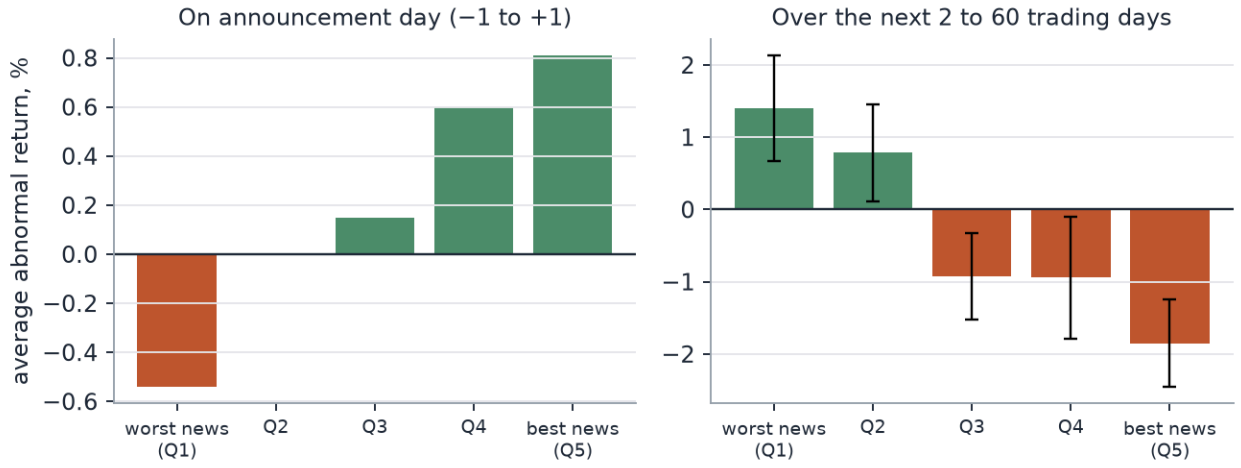


Figure 18. Left: the immediate reaction goes in the direction of the news. Right: the later drift goes the other way, and it gets steadily more negative as the news gets better.

My first reaction was that I had got a sign wrong somewhere, so I tested it. The announcement-day move has the right sign, so the surprise is not flipped. The reversal is there in 2000 to 2009 (−4.4% between best and worst news), 2010 to 2019 (−2.0%) and 2020 to 2026 (−4.3%), and it survives when I keep only events with no other filing nearby (−2.1%). It rises smoothly from group to group, which is not what noise looks like. The formal test, which uses a stricter surprise measure built from the press-release figures (3,825 reports), gives −2.6% with an interval from −3.5% to −1.7%. The chart above uses a broader sample of 9,359 reports, which is why its gap is larger, about −3.3%.

A fair objection is that good news should push a stock up, not down. It did, on the day. Here is how the best-news reports break down over the next two months:

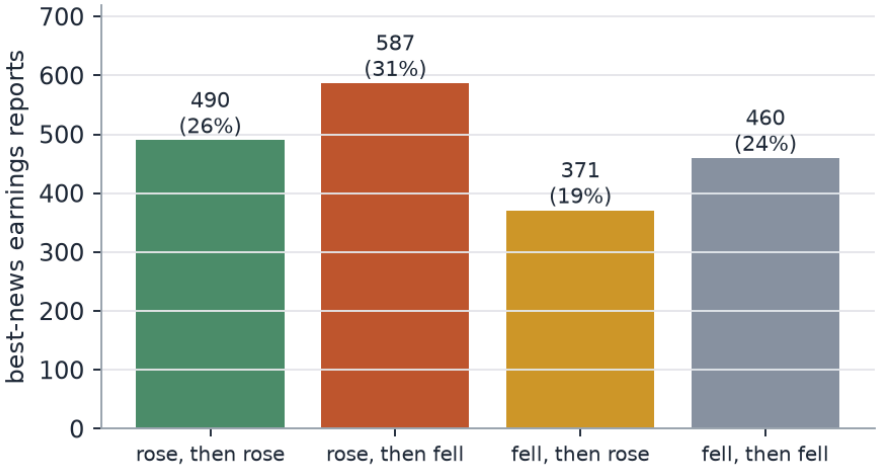


Figure 19. The 1,908 best-news earnings reports, split by what the stock did on the day and what it did over the next two months. 55% fell afterwards, and 31% rose first and then gave back about 10%.

So it is a tilt in the averages, not a rule: 45% of those stocks went on to rise. Why would it tilt this way? I have four candidate reasons and I did not test any of them. First, my surprise compares earnings with the same quarter last year, not with what analysts expected, so a result can beat last year and still disappoint. Second, prices can overreact to a headline and settle back, and the worst-news group shows the mirror image. Third, these are large, closely followed companies where good news is priced in within hours. Fourth, survivorship. Companies that got bad news and kept falling were often delisted, and they aren't in my data. That alone makes the worst-news group look better afterwards and could manufacture part of this pattern.

The wording I'm comfortable with is this. I did not find classic post-earnings drift in this survivor-only sample, and the data point the other way. That is a finding about this sample, not a refutation of the theory. To settle it I'd add delisted firms and use analyst forecasts for the surprise.

13. Could you trade it?

I added a simple backtest as a sanity check, not as a strategy. The signal is the probability of Up minus the probability of Down from the frozen model. Go long the top fifth, short the bottom fifth, enter at the first open after the decision time, and charge 10 basis points a side (20 in a second run), plus a borrow cost on shorts. I also ran a version with a one-session delay.

On the development years it looked pleasant: with the one-session delay the long book returned 11.8% a year, +3.7% a year ahead of the S&P 500 on the same days. On the locked test it fell apart. The long book earned 1.3% a year, the long-minus-index line -5.6% a year, and the hypothetical long-short -2.0%. Over the 3.6 years the long book gained about 4% while the index, on the same days, gained about 27%.

That is a useful lesson in itself. A good-looking result on the set you used to choose the model is not the same thing as a result on data you haven't touched, and the locked test did what it was there to do.

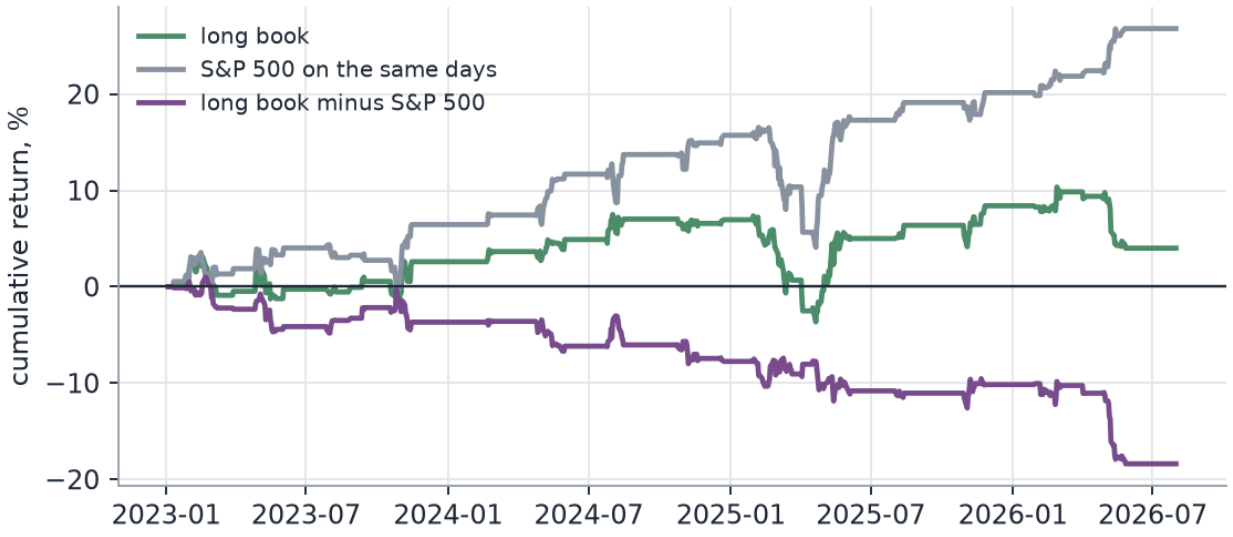


Figure 20. The locked test period, with no delay and 10 bps per side. Shorts are hypothetical, there is no capacity or delisting model, and this is a sanity check rather than a strategy.

14. Things that went wrong

I want this part to be useful, not apologetic, because it is where I learned the most. Besides the three data bugs in section 5, these are the ones I remember:

- **My own benchmark.** The first time I ran the GPU part on Colab it refused to start, saying the embeddings would take 19 GPU-hours against a 10-hour budget. The cause was my speed test: it timed a cold start, including the one-off cost of waking the GPU up. After adding a warm-up the projection was under an hour (the real run took 1.65 hours).
- **A regular expression that never finished.** A pattern for table rows had a nasty backtracking case, and three worker processes sat at full CPU for half an hour on one filing. I now run an adversarial timing test against every parsing pattern.
- **63 filings lost to server errors.** A burst of HTTP 503 responses from the SEC dropped 63 filings quietly. I added a retry pass that recovers them on the next run, and all 63 came back.
- **A p-value of zero.** In the drift-robustness check a p-value of exactly zero was being read as missing. That quietly marked a real effect as non-robust. Fixed, with a test.
- **The laptop went to sleep.** A long end-to-end test hung for two hours. Windows had put the machine into standby, and when it woke up the timeout fired. The test was fine, the setup was not.
- **Drive, and Streamlit.** Google Drive on Windows would not sync thousands of small files, so I uploaded zips through the browser. Getting the dashboard online took a second attempt: the first deployment stalled, so I added a requirements file listing only the six packages the dashboard needs instead of the whole project's.

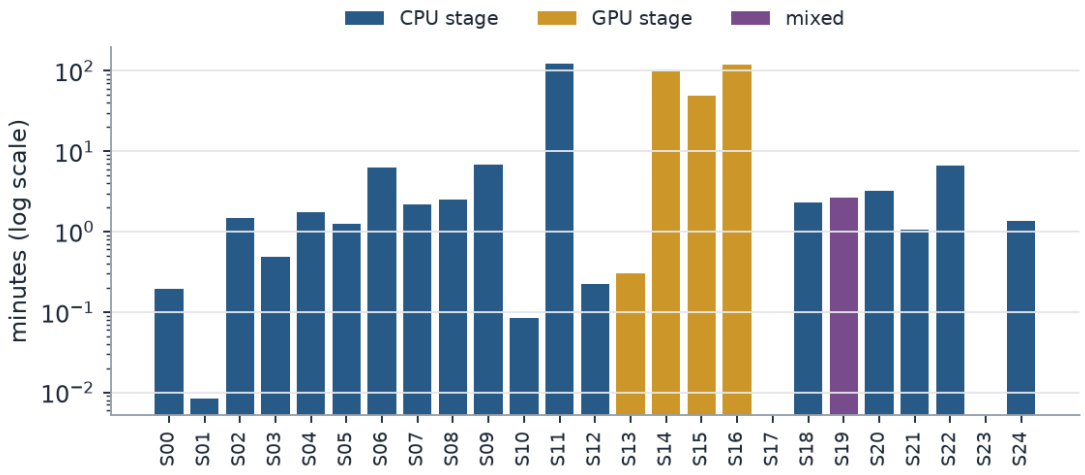


Figure 21. Wall-clock time per stage. The long bars are the tuning of the tree models (S11, two hours, on my laptop), the embeddings (S14) and the extraction run (S16).

The pipeline has 114 automated tests, and each of the problems above has one that fails on the old code.

15. What the results do and don't show

What I'm comfortable claiming. On 150 surviving S&P 500 stocks, with careful point-in-time controls, adding language-model embeddings of filing text to price, metadata and accounting features gave no out-of-sample gain. The models beat chance mostly by predicting *whether* a stock moves, not which way. The post-earnings drift reverses in this sample, with a survivorship caveat I can't remove.

What I'm not claiming. That text never helps. I only tested two encoders with small tuning budgets, on 150 big companies, with daily prices. I'm not claiming the reversal is a real market effect, or that anyone could trade on it, and the backtest says otherwise. And I'm not claiming the language-model extraction works, because I haven't scored it.

Known limits. The universe is survivors only, with daily bars only. The choice between the two top models was a statistical tie, though the rule froze the one with text: scored in the same run the model without text was slightly ahead, and LightGBM's predictions changed for about 20% of events when I re-scored the same model on a different machine. The firm-held-out and sector-held-out stress tests are built but I never ran their scores. The GPU model used on Colab was not recorded in the manifest. And everything about Track C is waiting for human labels.

16. What I'd do next

1. Run the 500-stock version, including non-S&P firms, on the corrected accounting code. Tighter intervals and a chance to compare index and non-index stocks.
2. Add delisted companies, which is the only real fix for the survivorship problem.

3. Label a few hundred filings by hand so the extraction can be scored, and then see whether extracted events beat plain embeddings.
4. Make LightGBM deterministic and compare the final candidates inside one run, preferring the strict model on a tie.
5. Run all four encoders, and consider real analyst forecasts for the earnings surprise.

17. Look at it yourself

Everything is public. The repository (https://github.com/harshatangirala/sec-filings-price-impact) has the pipeline, the configuration, the decision log (78 entries explaining each choice and bug), the notebook for Colab, the dashboard and its data, the automated tests, the slide deck, this report, and the analysis scripts behind the earnings-drift numbers. The interactive dashboard is at https://sec-filings-study.streamlit.app/. It walks the same story as a clickable flowchart of 16 steps with the charts behind each one, plus an 18-slide deck. If the app has gone to sleep, press the button and give it a minute.

To rerun it: open the notebook in Colab, set the profile and stock count in the first cell, and run all. Finished stages are skipped, so a disconnect costs nothing. You do not need any paid API or key, only a contact email for the SEC if you re-download.

Appendix: where each thing is

what	where
pipeline code	https://github.com/harshatangirala/sec-filings-price-impact/tree/main/src/secprice
decision log (every choice and bug)	https://github.com/harshatangirala/sec-filings-price-impact/blob/main/docs/decision_log.md
Colab notebook	https://github.com/harshatangirala/sec-filings-price-impact/tree/main/notebooks
dashboard code and data	https://github.com/harshatangirala/sec-filings-price-impact/tree/main/dashboard
automated report, figures, results tables	https://github.com/harshatangirala/sec-filings-price-impact/tree/main/reports
this report, the slide deck, and how I built them	https://github.com/harshatangirala/sec-filings-price-impact/tree/main/docs/write-up
earnings-drift analysis scripts	https://github.com/harshatangirala/sec-filings-price-impact/tree/main/analysis

Parameter counts are approximate, taken from model cards and file sizes. Nothing here is investment advice.